

EMG Pipeline Deep Dive — Mini-Book

Author: GitHub Copilot

1. Why This Document Exists

This guide explains *every major moving part* inside the EMG processing workflow shipped with `main_emg.py`. Think of it as your classroom companion: you can read it end-to-end to understand how EMG recordings travel from raw TXT files to tidy CSV metrics and plots, or you can jump to individual sections when you need to revisit a specific step. The goal is to make the codebase approachable—especially if you are a student or an analyst who wants to modify or extend the pipeline with confidence.

2. Bird's-Eye View of the Pipeline

```
main_emg.py
├─ build_day_index() → scan filesystem + metadata
├─ run_emg_pipeline() → orchestrate per-day processing
│   ├── load_day_acquisitions() → raw data loading + QC
│   ├── _process_day() → session-level preprocessing + metrics
│   │   ├── _compute_envelope() → EMG filtering + envelope
│   │   ├── compute_session_metrics()
│   │   ├── plotting helpers
│   │   └─ effort-bin logging
│   ├── _build_tables() → aggregate to daily/summary CSVs
│   ├── _plot_metric_trends()
│   └─ _write_effort_bins()
└─ quality report summary
```

Each block corresponds to a module or helper function. The subsections below dive into the specifics, moving from data discovery to final artifacts.

3. Entry Point: `main_emg.py`

EMG Pipeline Deep Dive — Mini-Book

Author: GitHub Copilot

This guide is intentionally long and detailed. The first half explains the architecture of the EMG pipeline. The second half walks step-by-step through a *worked example*: following one subject/day/session from raw MuscleBAN files through preprocessing, metric computation, and plotting. You can export this file to PDF and read it as a mini-book.

Part I – Architecture and Functions

1. Purpose and Scope

The EMG pipeline transforms raw MuscleBAN recordings into interpretable metrics and visuals. This mini-book explains the implementation in approachable terms, from high-level orchestration down to individual helper functions. It assumes basic familiarity with Python and NumPy/pandas, but not advanced signal processing background.

2. End-to-End Pipeline Overview

High-level flow:

1. Discover available subject/day folders using metadata.
2. Resolve per-device acquisition file paths and filter them.
3. Load raw data and enforce quality checks; skip unusable files.
4. Preprocess EMG (filtering + envelope), normalize to %MVC.
5. Compute per-session metrics and APDF.
6. Persist CSV tables (session/daily/changes/effort bins) and plots.

```
main_emg.py
├─ build_day_index()           # find DayAcquisition descriptors
├─ run_emg_pipeline()         # orchestrate per-day processing
│   ├─ load_day_acquisitions() # raw data loading + QC
│   ├─ _process_day()          # session-level preprocessing + metrics
│   │   ├─ _compute_envelope() # EMG filtering + envelope
│   │   ├─ compute_session_metrics() # APDF + scalar metrics
│   │   ├─ _record_effort_bins() # low/medium/high/%>100 effort
│   │   └─ plotting helpers    # APDF, histograms, grids, stacks
│   ├─ _build_tables()         # aggregate to daily/summary CSVs
│   ├─ _plot_metric_trends()
│   └─ _write_effort_bins()
└─ quality report summary
```

The rest of Part I goes module-by-module and function-by-function.

3. Entry Point and Configuration (`main_emg.py`)

3.1 Configuration constants

- `DATA_ROOT`: Root path containing `groupX/sensors/LIBPhys` #NNN subject data.
- `PARTICIPANTS_CSV`: Metadata CSV that maps subject IDs to groups, device numbers, and MACs.
- `SELECTED_SENSORS = {"MBAN": ["EMG"]}`: Instructs the loader to only keep EMG from MuscleBAN.
- `RESULTS_ROOT`, `PLOTS_ROOT`: Where CSVs and plots are written.
- `DEFAULT_CONFIG = PreprocessConfig()`: Default preprocessing parameters.

3.2 `_ensure_data_root()`

Purpose: guard rail before any heavy work begins.

Behavior:

1. Checks that `DATA_ROOT` exists on disk.
2. If missing, raises `FileNotFoundError` with a clear message instructing you to update the path.

This means: if you clone the repo on a different machine, this is usually the first place that fails until you point `DATA_ROOT` to your own data folder.

3.3 `build_day_index(...)` -> `list[DayAcquisition]`

Inputs:

- `subject_filter`: optional list of subject IDs (as strings) to include.
- `max_subjects`: upper bound on number of subjects.
- `max_days_per_subject`: cap on number of days per subject.

Steps:

1. Calls `_ensure_data_root()`.
2. Calls `discover_daily_acquisitions(DATA_ROOT, participants_csv=PARTICIPANTS_CSV, ...)`.
3. Receives a list of `DayAcquisition` objects.
4. Logs how many day folders and unique subjects it found.
5. Returns the list.

Why this design? It cleanly separates filesystem discovery from the rest of the code. You can unit test discovery alone, or reuse it in notebooks.

3.4 `main(...)`

Key flags:

- `run_all`: convenience flag that turns on `load_data = preprocess = visualize = True`.
- `load_data`: whether to scan the filesystem and build `DayAcquisition` list.
- `preprocess`: whether to actually run EMG processing.
- `visualize`: whether to generate plots (which can be slow).

Simplified pseudocode:

```
def main(run_all=False, load_data=False, preprocess=False, visualize=False, ...):
    if run_all:
        load_data = preprocess = visualize = True

    descriptors = []

    if load_data or preprocess:
        descriptors = build_day_index(...)
        if not descriptors:
            print("[main_emg] No acquisitions found. Nothing to do.")
            return None

    if not preprocess:
        print("[main_emg] Preprocessing skipped ...")
        return None
```

```

print("[main_emg] Starting EMG pipeline...")
quality_reports = []
artifacts = run_emg_pipeline(..., quality_log=quality_reports)

# summarise artifacts and quality
...
return artifacts

```

This function is what you call from the command line or from notebooks. The heavy lifting is delegated to `run_emg_pipeline`.

4. Dataset Enumeration and Loading (`load_signals`)

This package is responsible for everything that happens *before* signal processing: mapping metadata to folders, finding raw files, loading them into pandas, and applying basic quality checks and interpolation.

4.1 `meta_data.py`: Linking metadata to filesystem

Key functions

- `load_meta_data(csv_path='participants_info.csv') -> DataFrame`
 - Reads the CSV with `sep=';'` and `index_col='subject_id'`.
 - Returns a DataFrame where each row represents a subject. Typical columns include:
 - `group`: subject group (e.g. `G1`, `G2`).
 - `device_num`: numeric identifier used in the `LIBPhys ###` path.
 - `mBAN_left`, `mBAN_right`: MAC addresses of left/right MuscleBAN sensors.
- `get_muscleban_side(meta_data_df, mac_address) -> str | None`
 - Searches for a given MAC address across the left/right MAC columns.
 - Returns a string label (`mBAN_left` or `mBAN_right`).
 - Returns `None` if the MAC is not known in the metadata.

Why it matters

Without this mapping, filenames with MAC addresses would not be meaningful. This module gives semantic labels (left/right) that are later used in metrics and plots.

4.2 `dataset_loader.py`: Discovering day folders

`DayAcquisition` `dataclass`

Represents a single subject/day combination. Fields:

- `subject_id`: string, e.g., `'85'`.
- `group`: from metadata, used to locate the group folder.
- `device_num`: integer, used in `LIBPhys {device_num}`.
- `day_path`: `Path` to the day folder.
- `left_mac`, `right_mac`: MAC addresses for the MuscleBAN units.

Convenience properties:

- `date_label`: same as `day_path.name` (useful for plotting).
- `subject_root`: parent of `day_path` (the subject's root folder).

`discover_daily_acquisitions(...)`

Pseudocode structure:

```
def discover_daily_acquisitions(data_root, participants_csv, subject_filter=None,
...):
    meta = load_meta_data(participants_csv)
    results: list[DayAcquisition] = []

    for subject_id, row in meta.iterrows():
        if subject_filter and subject_id not in subject_filter:
            continue

        subject_root = (data_root / row['group'] / 'sensors' / f"LIBPhys
{row['device_num']}")
        if not subject_root.exists():
            # log and continue
            continue

        for day_folder in sorted(subject_root.iterdir()):
            if not day_folder.is_dir():
                continue
            results.append(DayAcquisition(..., day_path=day_folder, ...))

    return results
```

This is the bridge between *logical* metadata (subject IDs) and *physical* layout (folders on disk).

`load_day_acquisitions(day_descriptor, selected_sensors=None, quality_log=None)`

- Extracts `folder_path = day_descriptor.day_path`.
- Delegates to `load_daily_acquisitions` in `raw_data_loader.py`.
- Returns a nested dict structure: `device_label -> acquisition_label -> DataFrame`.

4.3 `path_handler.py`: How raw files are grouped

The main public function is `get_sensor_paths_per_device(folder_path, load_devices)`.

Parameters

- `folder_path`: a day folder, e.g. `.../LIBPhys 003/2025-10-01`.
- `load_devices`: dict mapping device names (`mбан`, `phone`, `watch`, etc.) to sensor lists.

Workflow

1. `_validate_load_devices(load_devices)`

- Checks for unknown device keys.
- Ensures sensor lists are non-empty and not duplicated.
- 2. For each device in `load_devices`:
 - Android devices (`phone`, `watch`, etc.): `_get_android_filepaths(...)` finds matching TSVs using patterns like `*ACC*`, `*GYRO*`.
 - `mban`: `_get_mban_files(...)` scans for MuscleBAN files and groups them by MAC.
- 3. For MBAN files:
 - `_group_mban_files(paths_dict)` uses metadata to convert MAC→side label.
- 4. `_group_files_by_acquisition(files)`
 - Groups files by their immediate parent folder (e.g. `09-15-00`, `MVC`).
- 5. `_keep_largest_file_per_acquisition(grouped)`
 - When multiple candidates exist for a given slot, keeps the largest file.
 - For MVC slots, calls `_select_preferred_mban_file` which:
 - Searches only for filenames whose stem includes `OSCompatible` (case-insensitive).
 - Returns `None` if no `OSCompatible` file is present, effectively disabling that MVC.

Mental model

Think of `path_handler` as building a *routing table*:

```
mban_left:
  MVC:      /path/to/MVC/StudioData..._OSCompatible.txt
  09-15-00: /path/to/09-15-00/StudioData...txt
  10-30-00: /path/to/10-30-00/StudioData...txt
mban_right:
  ...
```

This table is then consumed by the loader to actually open the files.

4.4 `raw_data_loader.py`: Loading and cleaning

`load_daily_acquisitions(folder_path, load_devices, fs_android, padding_type, quality_log)`

1. Calls `get_sensor_paths_per_device` to obtain the routing table.
2. Initializes an empty nested dict, e.g. `{'mban_left': {}, 'mban_right': {}}`.
3. Iterates devices and acquisitions:
 - For MBAN devices:
 - Picks the single path selected earlier.
 - Calls `_load_muscleban_data(path, sensor_list)` in a `try` block.
 - On `DataQualityError`, appends its report to `quality_log` and continues.
 - For Android devices:
 - Calls `_load_raw_data(acquisition_paths)` to read multiple sensor files.
 - Calls `_pad_data` to align sensor start/end times.
 - Calls `_re_sample_data` / interpolation helpers to a common sampling rate.
4. After loading all devices, calls `_create_loading_report` to print which sensors were successfully loaded for which acquisitions.

5. Returns the nested dict.

`_load_muscleban_data(file_path, sensor_list)` in more detail

Steps on a typical EMG file:

1. Read the TSV using pandas.
2. Drop columns that are entirely NaN.
3. Drop a known constant zero column (firmware artifact), if present.
4. When the last three columns look like `MAG` channels, drop them as they are not needed.
5. Rename the remaining columns to match `VALID_MBAN_DATA = ["nSeq", "EMG", "xACC", "yACC", "zACC"]`.
6. Detect whether this is an `OSCompatible` MVC:
 - Parent folder is named `MVC`.
 - Filename stem contains `OSCompatible` (case-insensitive).
7. Depending on that detection, pick `min_samples`:
 - `OSCompatible` MVC → `MIN_MVC_OSCOMPATIBLE_SAMPLES` (8 seconds at 1000 Hz).
 - Regular session → `MIN_MUSCLEBAN_SAMPLES` (30 seconds at 1000 Hz).
8. Call `assess_muscleban_dataframe(df, file_path, min_samples)`.
 - If it returns a report flagged as failing, raises `DataQualityError`.
9. Select only columns corresponding to requested sensors (e.g. `EMG` only) plus `nSeq`.

This gives you a clean, validated EMG time series ready for interpolation or direct preprocessing.

Other helpers

- `_load_sensor_file(file_path, sensor_name)` for Android logs and reads a single TSV, renaming columns to a consistent schema.
- `_pad_data(sensor_data, report, padding_type)` ensures all sensors for one acquisition share a common time window by trimming or padding.
- `_re_sample_data(...)` in conjunction with `interpolate.py` functions resamples signals to `fs_android`.

5. Signal Processing Orchestration (`emg_pipeline.py`)

`PreprocessConfig` records all the tunable parameters for EMG preprocessing:

- `fs` — sampling rate (e.g. 1000 Hz).
- `lowcut, highcut` — band-pass frequencies used before enveloping.
- `smooth_sigma_ms` — smoothing scale used in the Gaussian filter(s) inside `preprocess_emg`.
- `envelope_preview_seconds` — how many seconds of data to overlay in envelope plots.

`run_emg_pipeline(day_descriptors, selected_sensors, results_root, plots_root, config, percentiles, generate_visuals, quality_log)`:

1. Creates `results_root` and `plots_root` if needed.
2. Uses `quality_log` (list) to collect any new `FileQualityReport` objects from loading.
3. For each `DayAcquisition` in `day_descriptors`:
 - `load_day_acquisitions(day, selected_sensors, quality_log)` returns nested dict.
 - If dict is empty, logs and continues.

- Calls `_process_day(day, day_data, config, percentiles, plots_root_if_any, effort_records)`.
- 4. After the loop:
 - Persists `quality_records` via `_persist_quality_report` (if not empty).
 - Persists `effort_records` via `_write_effort_bins` (if not empty).
- 5. If `session_metrics` is empty, returns early with whatever artifacts were created.
- 6. Builds DataFrames, writes them to CSV, and optionally builds trend plots.

`_process_day(day, day_data, config, percentiles, plots_root, effort_records):`

1. Initializes `day_metrics` and `daily_payload`.
2. Loops over `device_label, acquisitions` in `day_data.items()`:
 - Uses `_pick_mvc(acquisitions)` to find MVC.
 - Computes MVC envelope with `_compute_envelope`, checks `mvc_peak > 0`.
 - For each non-MVC `session_label`:
 - Calls `_compute_envelope(session_df, config, return_raw=True)`.
 - Converts to %MVC using `mvc_peak`.
 - Builds metadata (`_build_metadata`).
 - Calls `compute_session_metrics`, appends metrics dict.
 - Calls `_record_effort_bins` to append a row for the effort table.
 - If `plots_root` is provided, calls `_save_session_visuals`.
 - Adds `(percent_signal.copy(), fs)` to `daily_payload` keyed by `(side_label, session_label)`.
3. After loops: if `plots_root` and `daily_payload`, calls `_save_day_visuals`.
4. Returns `day_metrics`.

Important detail: %MVC normalization uses the *peak* of the MVC envelope, which ensures that 100% corresponds to the subject's individual maximum contraction.

6. Metrics and Visuals (`emg_analysis`)

`compute_apdf(signal_percent, percentiles):`

- Sorts `signal_percent` values.
- Creates a probability axis from 0 to 100.
- Computes amplitudes at given probability levels (e.g., P10, P50, P90).

`compute_session_metrics(signal_percent, fs, metadata, percentiles):`

- Computes:
 - iEMG in %MVC·seconds.
 - Mean/min/max/median %MVC.
 - APDF percentiles.
 - Duration and sample count.
- Returns metrics dict + APDFResult.

Visualization helpers (`visuals.py`) convert metrics and %MVC arrays into figures:

- APDF curves and histograms.
- Time-series trends for percentage changes.

- Effort grids and stacks for left/right and session comparisons.

Part II – Worked Example: One of *Your* Subjects

This part is narrative rather than purely API-focused. It explains what happens internally when **you** run the pipeline on your dataset.

Below we use concrete identifiers that match your repository setup:

- The main entry point you ran is:

```
C:/Users/gonba/PrevOccupAI_Plus/EMG_venv/Scripts/python.exe main_emg.py
```

- The metadata file is `participants_info.csv` in the repo root.
- The EMG-only configuration uses `SELECTED_SENSORS = {MBAN: ["EMG"]}`.

The exact subject IDs and dates in your data may differ; adapt the IDs below to a real subject/day you care about (e.g., `subject_id='001'`, date folder `2024-03-14`). The steps remain identical.

7. Choosing a Subject and Day

When you call `main(run_all=True, ...)`, the pipeline does:

1. `load_meta_data('participants_info.csv')` to obtain one row per subject.
2. For a particular subject (say `subject_id='001'`):
 - It reads the `group` (e.g. `G1`) and `device_num` (e.g. `3`).
 - It constructs a path like:

```
<DATA_ROOT>/G1/sensors/LIBPhys 003/
```

3. Inside that folder it looks for day subfolders, such as:

```
2024-03-14/  
2024-03-21/
```

4. It creates a `DayAcquisition` for each day. We'll focus on one, for example:

```
<DATA_ROOT>/G1/sensors/LIBPhys 003/2024-03-14
```

8. From Filenames to Devices and Sides

Within 2024-03-14, you might have acquisition folders like:

```
09-15-00/  
10-30-00/  
MVC/
```

Each contains one or more TSV files whose names encode a MAC address, for example:

```
09-15-00/StudioData_003_84FD27E50653_20240314_091500.txt  
09-15-00/StudioData_003_84FD27E4A1B2_20240314_091500.txt  
MVC/StudioData_003_84FD27E50653_20240314_100000_OSCCompatible.txt  
MVC/StudioData_003_84FD27E4A1B2_20240314_100000_OSCCompatible.txt
```

`get_sensor_paths_per_device` does the following:

- Extracts MAC addresses from each filename.
- Looks up each MAC in `participants_info.csv` via `get_muscleban_side`.
- Assigns them to `mBAN_left` or `mBAN_right`.
- Groups the files by acquisition folder (09-15-00, MVC, etc.).

For MVC specifically:

- `_select_preferred_mban_file` filters candidates to those whose stem includes `OSCompatible`.
- It keeps the largest such file per side.
- If no `OSCompatible` file exists, MVC for that side is *ignored*, which will later cause `_pick_mvc` to skip that device.

9. Loading One MVC and One Session (Your Data)

Suppose your `mBAN_left` device has:

- An `OSCompatible` MVC file under `MVC/`.
- A normal session under `09-15-00/`.

For the MVC:

1. `load_daily_acquisitions` calls `_load_muscleban_data(mvc_path, ["EMG"])`.
2. `_load_muscleban_data` reads the TSV, cleans columns, renames them, and detects:
 - Parent folder is `MVC`.
 - Filename contains `OSCompatible`.
 - So it sets `min_samples = MIN_MVC_OSCOMPATIBLE_SAMPLES` (8000 samples).
3. `assess_muscleban_dataframe` validates that:
 - Sample count ≥ 8000 .
 - EMG column exists.
 - NaN ratio and zero ratio are below configured thresholds.
4. If all checks pass, the resulting DataFrame has at least `nSeq` and `EMG` columns.

For the 09-15-00 session:

1. `_load_muscleban_data(session_path, ["EMG"])` is called.
2. Because this is *not* an MVC, it uses `MIN_MUSCLEBAN_SAMPLES` (30000 samples) as minimum.
3. The same quality checks are applied, but the minimum duration is much longer (30 seconds).

At the end of loading, `load_daily_acquisitions` returns a nested dict similar to:

```
{
  'mBAN_left': {
    'MVC': mvc_df_left,
    '09-15-00': session_df_left,
  },
  'mBAN_right': {
    'MVC': mvc_df_right,
    '09-15-00': session_df_right,
  },
}
```

10. Preprocessing to EMG Envelope

For each device and each acquisition, `_process_day` calls `_compute_envelope(df, config, return_raw=False or True)`.

`_compute_envelope` does:

1. `_extract_emg_mv(df)` to pick out the EMG channel and convert it to millivolts.
2. `preprocess_emg` (from `emg_analysis.preprocessing`) to:
 - Apply a band-pass filter in the EMG range.
 - Rectify and smooth the signal to obtain an EMG envelope.
3. Returns:
 - Envelope only (for MVC).
 - Raw EMG + envelope (for regular sessions) if `return_raw=True`.

For your MVC acquisition on `mBAN_left`, `_compute_envelope(mvc_df_left, config)` yields an array `mvc_env_left`. The code then computes:

```
mvc_peak_left = mvc_env_left.max()
```

For the 09-15-00 session:

```
envelope_left, raw_left = _compute_envelope(session_df_left, config,
return_raw=True)
```

11. Normalizing to %MVC and Computing Metrics

Now we normalize the session envelope to %MVC:

```
percent_signal_left = (envelope_left / mvc_peak_left) * 100.0
```

Interpretation on your data:

- Samples where `envelope_left` equals `mvc_peak_left` correspond to 100% MVC.
- Values above 100% indicate activity higher than the chosen MVC peak (e.g., noisy bursts or slightly stronger contractions).

`compute_session_metrics(percent_signal_left, fs=config.fs, metadata=meta_left, percentiles=(10, 50, 90))` then computes:

- Integrated EMG in %MVC-seconds.
- Mean, min, max, and median %MVC.
- APDF P10, P50, P90.
- Duration and sample count.

These results form one row in `session_metrics.csv` with columns like:

- `subject_id`, `date`, `side`, `device`, `session_label`.
- `mean_emg_percent`, `p50_emg_percent`, `iemg`, etc.

12. Effort Bins for Your Session

Immediately after computing metrics, `_process_day` calls `_record_effort_bins(effort_records, metadata, percent_signal_left, fs)`.

`_record_effort_bins` internally calls `_compute_effort_bins(percent_signal_left, fs)`:

1. Uses `EFFORT_BANDS` to define ranges (e.g. 0–33, 33–66, 66–100).
2. Counts how many samples fall in each band.
3. Converts to minutes: `minutes = count / fs / 60`.
4. Converts to percentages of total samples: `pct = 100 * count / total_samples`.

Suppose, for your `09-15-00` session, the result is:

- 20 minutes and 40% in Low effort.
- 22 minutes and 45% in Moderate.
- 7 minutes and 15% in High.
- 0 minutes and 0% in >100%.

These values (actual numbers will differ) are written into `session_effort_bins.csv` as columns like:

- `low_effort_minutes`, `low_effort_pct`.
- `moderate_effort_minutes`, `moderate_effort_pct`.
- `high_effort_minutes`, `high_effort_pct`.
- `above_100_effort_minutes`, `above_100_effort_pct`.

Together with identifying metadata for that row:

- `subject_id='001'`.
- `date='2024-03-14'`.
- `side='left'`.
- `session_label='09-15-00'`.

13. Visualizing That Session

If `visualize=True`, `_save_session_visuals` writes plots into:

```
results/emg_pipeline/plots/001/2024-03-14/left/09-15-00/
```

Typical contents:

- `09-15-00_apdf.png`: APDF curve with highlighted P10/P50/P90 for your session.
- `09-15-00_hist.png`: histogram of %MVC amplitudes.
- `09-15-00_envelope.png`: preview showing raw EMG vs envelope for a few seconds.

You can open these in any image viewer to quickly inspect data quality for that specific acquisition.

14. Day Summary

After all sessions for that day are processed, `_save_day_visuals` uses `daily_payload` to generate summaries.

For `subject_id='001'`, `date='2024-03-14'`, it creates:

```
results/emg_pipeline/plots/001/2024-03-14/summary/
```

Inside you'll find, for example:

- `effort_grid.png`: matrix of bar charts; rows = sessions, columns = `left/right`.
- `effort_stacks.png`: stacked horizontal bars for each session showing Low/Moderate/High/%>100.

This makes it easy to visually compare multiple sessions across the same working day.

15. Data Tables for Analysis

Finally, back in `run_emg_pipeline`, `_build_tables` and `_write_tables` construct and save:

- `session_metrics.csv`: one row per session/side; used for detailed analysis.
- `daily_metrics.csv`: aggregated per day/side statistics.
- `session_increments.csv`: percentage changes between consecutive sessions on the same day.
- `daily_increments.csv`: percentage changes between consecutive days.

These CSVs appear under `results/emg_pipeline/` and are directly loadable in pandas or Excel. For example, you can do:

```
import pandas as pd

sessions = pd.read_csv("results/emg_pipeline/session_metrics.csv")
print(sessions.head())
```

to quickly inspect the computed metrics for your cohort.

Part III – Export, Debugging, and Next Steps

16. Export to PDF

You can export this Markdown to PDF using Pandoc or VS Code extensions.

With Pandoc installed:

```
pandoc "docs/emg_pipeline_overview.md" -o "docs/emg_pipeline_overview.pdf"
```

Or, install “Markdown PDF” extension in VS Code and use its “Export (pdf)” command on `docs/emg_pipeline_overview.md`.

17. Glossary

- **EMG:** Electromyography.
- **MVC:** Maximal Voluntary Contraction.
- **APDF:** Amplitude Probability Distribution Function.
- **iEMG:** Integrated EMG (area under the curve).
- **%MVC:** EMG amplitude normalized to the MVC peak.
- **OSCompatible:** Special MVC recording export format aligned for OpenSignals compatibility.
- **Plot tweaks:** All plotting behavior lives in `emg_analysis.visuals`. You can easily alter color schemes, layout, or add new plot types.
- **Quality heuristics:** Modify `assess_muscleban_dataframe()` to add new checks (e.g., frequency content, saturation detection).

11. Suggested Study Path

1. **Run the pipeline** (`python main_emg.py`) with a single subject/day subset.
2. **Open `session_effort_bins.csv` and plots** for that run; correlate numbers with visuals.
3. **Read `emg_pipeline.py`** alongside this document to see how the narrative maps to implementation.
4. **Experiment:** tweak `PreprocessConfig`, run again, and observe how metrics/plots change.
5. **Trace quality failures** via `data_quality_report.csv` to understand common data issues.

12. Final Thoughts

The EMG pipeline is modular by design: loaders handle filesystem quirks, signal processing modules focus on physics/biomechanics, and analytics modules produce actionable summaries. With the combination of code

docstrings (already added across modules) and this high-level tour, you should feel comfortable reading, debugging, and extending the system.

If you ever need deeper explanations (e.g., filter design rationale, biomechanical interpretation of metrics), feel free to reach out or file documentation requests—this guide is meant to evolve with your questions.